



TECHNICAL SPECIFICATION v1.1

AI Harness

Гибридная спецификация v1.1
С правками по архитектурному аудиту

Изменения: AST-валидатор, dependency-cruiser, Circuit Breaker, JSON Schema

Модули: 8 (L1: 2, L2: 2, L3: 2, Cross: 2)

Версия: 1.1 (ревью от 02.06.2026)

Июнь 2026

Содержание

- 0. Сводка правок по ревью**
- 1. Принципы гибридного подхода**
- 2. Архитектура: 8 модулей**
- 3. L1: Context System**
 - 3.1. Модуль: AGENTS.md
 - 3.2. Модуль: Instruction Modules
- 4. L2: Quality & Safety**
 - 4.1. Модуль: Quality Pipeline
 - 4.2. Модуль: Safety Config
- 5. L3: Memory & Protocol**
 - 5.1. Модуль: Knowledge Base
 - 5.2. Модуль: Session Protocol
- 6. Cross-cutting: Feedback**
 - 6.1. Модуль: Skills Registry
 - 6.2. Модуль: Feedback Loop
- 7. Deterministic Validator (переписан)**
- 8. JSON Schema контракты**
- 9. Манифест (.harness/manifest.json)**
- 10. Три режима работы**
- 11. Пример: полный SKILL.md**

0. Сводка правок по ревью

Настоящий документ версии 1.1 содержит правки по архитектурному аудиту v1.0. Ниже сводка четырех критических точек и их решений.

Таблица 0: Матрица правок по ревью

Точка	Проблема (v1.0)	Решение (v1.1)	Где в документе
1. Хрупкость валидации	grep/wc/sed для Markdown — падают на локализации, пробелах, регистре	Node.js + remark-parse Проверка узлов, не строк.	AST. Section 7
2. Нет контроля слоев	"UI не импортирует data-access" — текст, не код. Нарушение не ловится.	dependency-cruiser .dependency-cruiser.js. Architecture-as-Code.	с Section 4.1, 3.1
3. Deadlock в Feedback Loop	Агент бесконечно пытается починить lint, тратит токены, ломает код.	Circuit ATTEMPTS_LIMIT=3, triage_report.md + handoff.	Breaker: Section 6.2 затем
4. Слабая валидация frontmatter	head -1 grep "----" проверяет только разделитель, не содержимое.	JSON Schema для feature_list.json и frontmatter через ajv-cli.	Section 8

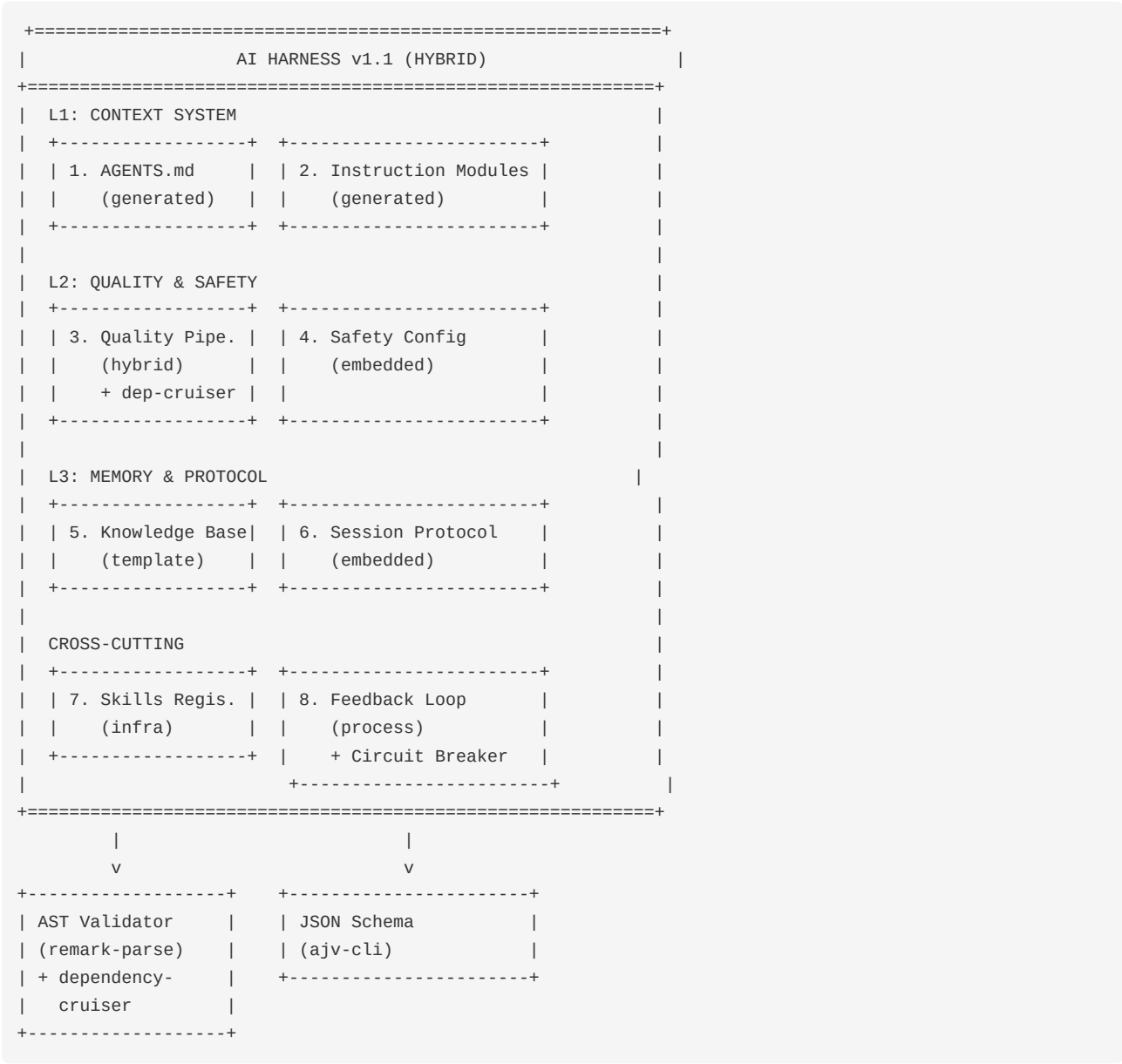
1. Принципы гибридного подхода

Таблица 1: Правила гибридного подхода

#	Правило	Смысл
1	Skill описывает, агент генерирует	SKILL.md содержит декларативные описания модулей. Агент читает skill, анализирует проект, генерирует артефакты.
2	Security-критичное копируется, не генерируется	CSP, secrets, auth-конфиги — verified templates. Агент адаптирует переменные, но структура фиксирована.
3	Все валидируется AST-based checker 'ом	validate-harness.ts (remark-parse) проверяет структуру Markdown AST. dependency-cruiser проверяет архитектурные границы. JSON Schema валидирует метаданные. Никакого grep.
4	Circuit Breaker для Feedback Loop	ATTEMPTS_LIMIT=3. После — triage_report.md и human handoff. Никаких бесконечных циклов.

- Generated** — агент генерирует на основе skill и контекста проекта
- Template** — копируется из verified template с минимальной адаптацией
- Embedded** — встроено в другой модуль, не существует отдельно

2. Архитектура: 8 модулей



3. L1: Context System

3.1. Модуль: AGENTS.md (Root Manifest)

Тип: **Generated** Артефакт: `./AGENTS.md` Слой: L1

Назначение

Корневой файл контекста. Агент читает при каждом старте. Содержит стек, архитектурные решения, команды, антипаттерны (теперь через dependency-cruiser), safety-конфигурацию, session protocol, feedback loop.

PRAVKA по revyu #2: Architecture-as-Code

Было (v1.0): Anti-patterns как текст: "No cross-layer imports (UI cannot import from data-access)"

Стало (v1.1): Anti-patterns как код. dependency-cruiser (.dependency-cruiser.js) с жесткими ограничениями импортов. AGENTS.md ссылается на конфиг, а не декларирует текстом.

SKILL.md секция

```
## Модуль: AGENTS.md
НЕОБХОДИМО: AGENTS.md в корне проекта, < 50 строк.
ШАБЛОН: template/AGENTS.md

СТРУКТУРА:

## Stack
[из package.json: Next.js X, React Y, TS Z, ...]

## Architecture
[layered | monolith | mfe – auto-detect from structure]

## Commands
[из scripts: dev, build, test, lint, typecheck]

## Architecture Rules (dependency-cruiser)
Enforced by .dependency-cruiser.js – see config for rules.
Key: UI layer cannot import from data-access layer.

## Safety
These operations require explicit user confirmation:
- rm -rf, git push --force, curl | sh
- Editing .env files
- Modifying CSP or security headers
- Installing packages with postinstall scripts
- Modifying database schemas with data loss risk

## Session Protocol
1. Before starting: read feature_list.json
2. Per feature: conventional commit ("feat: F02 title")
3. On context switch: /clear + pass structured context
4. On completion: update feature_list.json

## Feedback Loop
When pipeline catches mistake: record Lesson, update instruction.
Circuit Breaker: max 3 attempts, then triage_report.md + human.

ВОПРОСЫ ПОЛЬЗОВАТЕЛЮ:
1. "Какая архитектура проекта?" (если не определяется)
2. "Есть ли особые требования к безопасности?"

ВАЛИДАЦИЯ (AST-based, НЕ grep):
- remark-parse: AST содержит heading "Stack", "Commands",
  "Architecture Rules", "Safety", "Session Protocol"
- Размер < 60 строк (wc -l на результирующем тексте)
- .dependency-cruiser.js существует и содержит forbidden-правила
```

Пример сгенерированного AGENTS.md

```
## Stack
Next.js 15, React 19, TypeScript 5.6, Tailwind CSS, Prisma, PostgreSQL

## Architecture
Layered: UI -> Services -> Data Access.
See .dependency-cruiser.js for enforced boundaries.

## Commands
- pnpm dev          # Development server
- pnpm build        # Production build
- pnpm test         # Vitest unit tests
- pnpm lint         # Biome check + depcruise
- pnpm typecheck    # tsc --noEmit

## Architecture Rules
Enforced by .dependency-cruiser.js:
- src/ui/** cannot import from src/data-access/**
- src/services/** can import from data-access, not from ui
- Cross-layer imports are forbidden at CI level

## Safety
These operations require explicit user confirmation:
- rm -rf, git push --force, curl | sh
- Editing .env, .env.local files
- Modifying CSP or security headers
- Installing packages with postinstall scripts
- Modifying database schemas with data loss risk

## Session Protocol
1. Before starting: read feature_list.json
2. Per feature: conventional commit ("feat: F02 image pipeline")
3. On context switch: /clear + pass structured context
4. On completion: update feature_list.json + record lesson if needed

## Feedback Loop
When pipeline catches mistake: record Lesson in docs/knowledge/,
update relevant instruction module.
Circuit Breaker: 3 attempts max, then triage_report.md.
```

Template: .dependency-cruiser.js

```
/** @type {import('dependency-cruiser').IConfiguration} */
module.exports = {
  forbidden: [
    {
      name: 'ui-cannot-import-data-access',
      comment: 'UI layer must not import from data-access directly',
      severity: 'error',
      from: { path: '^src/ui' },
      to: { path: '^src/data-access' }
    },
    {
      name: 'ui-cannot-import-services-internally',
      comment: 'UI can only import from public service API',
      severity: 'warn',
      from: { path: '^src/ui' },
      to: { path: '^src/services', pathNot: '^src/services/index\\.ts$' }
    }
  ],
  options: {
    doNotFollow: { path: 'node_modules' },
    tsConfig: { fileName: 'tsconfig.json' }
  }
};
```

3.2. Модуль: Instruction Modules

Тип: **Generated**

Артефакт: `./.claude/instructions/` Слой: L1

Назначение

Объединение Path Rules и Skills. Два типа: `local/` (project-specific, агент генерирует) и `shared@version/` (cross-project, git subtree).

SKILL.md секция

```
## Модуль: Instruction Modules
НЕОБХОДИМО: .claude/instructions/local/ и .claude/instructions/shared/

LOCAL (генерируй на основе стека):
- react-components.md (если React)
- api-endpoints.md (если backend routes)
- tests.md (всегда)

SHARED (подключай):
git subtree add --prefix=.claude/instructions/shared \
  https://github.com/your-org/harness-instructions.git v1.0.0

ФОРМАТ:
---
name: [name]
type: local | shared
trigger: glob:**/*.tsx | phase:w2 | manual
tags: [react, ui, testing]
---
# [Title]
## Context
## Rules
## Examples

ВАЛИДАЦИЯ (AST + Schema):
1. remark-parse: каждый .md имеет frontmatter блок (---)
2. JSON Schema: type enum ["local", "shared"], tags — массив строк
```

4. L2: Quality & Safety

4.1. Модуль: Quality Pipeline

Тип: **Generated** + **Template**

Артефакт: `lefthook.yml`, `.dependency-cruiser.js`, `biome.json`, `.github/workflows/ci.yml`

Слой: L2

Назначение

Объединение Lefthook, Custom Linters и CI Gates + **dependency-cruiser** для архитектурного контроля. Единый pipeline: lint + typecheck + test + architecture-check.

PРАВКА по revyu #2: Architecture-as-Code

Добавлено: `dependency-cruiser` (`.dependency-cruiser.js`) в Quality Pipeline. Шаг "Architecture Check" в lefthook и CI. Границы слоев — код, не текст.

SKILL.md секция

```
## Модуль: Quality Pipeline
НЕОБХОДИМО: lefthook.yml + .dependency-cruiser.js + .github/workflows/ci.yml

ВЫБОР ИНСТРУМЕНТОВ:
TypeScript -> Biome + ESLint + tsc + vitest + dependency-cruiser
Python -> ruff + мпы + pytest
Go -> golangci-lint + go test

LOCAL (lefthook.yml):
pre-commit:
  parallel: true
  commands:
    lint: {LINT_CMD}
    typecheck: {TYPECHECK_CMD}
    test: {TEST_CMD}
    architecture: depcruise src # <-- добавлено в v1.1

CI (.github/workflows/ci.yml):
jobs: lint, typecheck, test, architecture, build
architecture job: npx depcruise src --config .dependency-cruiser.js

TEMPLATE: lefthook.yml и ci.yml — из template/, агент заменяет {VARIABLES}

ВАЛИДАЦИЯ:
test -f lefthook.yml && 3+ команды
test -f .dependency-cruiser.js && содержит forbidden[]
test -f .github/workflows/ci.yml && 4+ jobs (включая architecture)
```

Updated lefthook.yml (c architecture check)

```
# template/lefthook.yml (v1.1)
pre-commit:
  parallel: true
  commands:
    lint:
      run: {LINT_CMD}
    typecheck:
      run: {TYPECHECK_CMD}
    test:
      run: {TEST_CMD}
    architecture:
      run: npx depcruise src --config .dependency-cruiser.js

pre-push:
  commands:
    build:
      run: {BUILD_CMD}
```

4.2. Модуль: Safety Config

Тип: **Embedded** Артефакт: Раздел `## Safety` в AGENTS.md Слой: L2

Назначение

Блокировка опасных операций — раздел в AGENTS.md. Не генерируется агентом, фиксирован.

SKILL.md секция

```
## Модуль: Safety Config
ТИП: Embedded (раздел ## Safety в AGENTS.md)
СОДЕРЖАНИЕ (фиксировано):
  These operations require explicit user confirmation:
  - rm -rf, git push --force, curl | sh
  - Editing .env, .env.local, .env.production files
  - Modifying CSP headers, CORS config, auth middleware
  - Installing packages with postinstall scripts
  - Exposing ports, changing firewall rules
  - Modifying database schemas with data loss risk

ВАЛИДАЦИЯ: AST check — раздел ## Safety с 5+ list items.
```

5. L3: Memory & Protocol

5.1. Модуль: Knowledge Base

Тип: **Template** Артефакт: docs/knowledge/ Слой: L3

Назначение

Объединение ADR и Lessons. Единая директория с тегируемыми записями типов: `decision`, `lesson`, `pattern`.

SKILL.md секция

```
## Модуль: Knowledge Base
НЕОБХОДИМО: docs/knowledge/ с шаблонами и JSON Schema.

ШАБЛОНЫ (копируются из template/knowledge/):
.template-decision.md, .template-lesson.md, .template-pattern.md

JSON SCHEMA (docs/knowledge/.schema.json):
Валидация frontmatter: type enum["decision","lesson","pattern"],
tags — массив строк, date — ISO формат, severity enum["high","medium","low"]

ВАЛИДАЦИЯ:
1. Шаблоны на месте
2. JSON Schema проходит ajv-cli проверку
3. Каждый .md (кроме шаблонов) — валидный frontmatter по схеме
```

5.2. Модуль: Session Protocol

Тип: **Embedded** Артефакт: Раздел в AGENTS.md + feature_list.json Слой: L3

Назначение

Объединение Git Checkpoints и Context Reset Protocol.

SKILL.md секция

```
## Модуль: Session Protocol
ТИП: Embedded + feature_list.json
```

```
JSON SCHEMA (feature_list.schema.json):
```

```
{
  "type": "object",
  "required": ["project", "features", "tech_debt", "last_updated"],
  "properties": {
    "project": { "type": "string" },
    "features": {
      "type": "array",
      "items": {
        "type": "object",
        "required": ["id", "title", "status"],
        "properties": {
          "id": { "type": "string", "pattern": "^F[0-9]{2}$" },
          "title": { "type": "string", "minLength": 3 },
          "status": { "enum": ["pending", "in_progress", "completed"] },
          "commit": { "type": "string" }
        }
      }
    },
    "tech_debt": { "type": "array", "items": { "type": "string" } },
    "active_decisions": { "type": "array", "items": { "type": "string" } },
    "last_updated": { "type": "string", "format": "date" }
  }
}
```

```
ВАЛИДАЦИЯ: ajv-cli validate --schema=feature_list.schema.json feature_list.json
```

6. Cross-cutting: Feedback

6.1. Модуль: Skills Registry

Тип: **Template** Артефакт: `~/harness/instructions/shared/` или git repo Слой: Cross

Назначение

Центральный репозиторий shared-инструкций. Инфраструктурный модуль.

SKILL.md секция

```
## Модуль: Skills Registry
ТИП: Infrastructure

ПОДКЛЮЧЕНИЕ:
git subtree add --prefix=.claude/instructions/shared \
  https://github.com/your-org/harness-instructions.git v1.0.0

ВЕРСИОНИРОВАНИЕ: semver tags
ОБНОВЛЕНИЕ: git subtree pull
```

6.2. Модуль: Feedback Loop

Тип: **Generated** Артефакт: Процесс, описанный в AGENTS.md Слой: Cross

Назначение

Объединение Improvement Loop и Evaluator. Цикл: агент ошибается -> pipeline ловит -> lesson -> instruction обновляется.

PRAVKA по revyu #3: Circuit Breaker

Было (v1.0): Агент бесконечно пытается починить ошибку pipeline.

Стало (v1.1): Circuit Breaker — ATTEMPTS_LIMIT=3. После 3-й неудачи: создание triage_report.md с описанием конфликта и передача управления человеку.

SKILL.md секция

```
## Модуль: Feedback Loop
ТИП: Process (раздел в AGENTS.md)

ЦИКЛ:
1. Agent makes mistake (uses `any`, forgets test, etc.)
2. Quality Pipeline catches it (lint fails, test fails, depcruise fails)
3. Agent records Lesson in docs/knowledge/NNN-slug.md
4. Agent updates relevant instruction module
5. On next session: Agent reads updated instructions

CIRCUIT BREAKER (v1.1):
ATTEMPTS_LIMIT = 3
Поведение:
  Попытка 1: pipeline fails -> исправь -> recommit
  Попытка 2: pipeline fails -> исправь -> recommit
  Попытка 3: pipeline fails -> создай triage_report.md -> STOP

triage_report.md содержит:
- Что сломалось (ошибка pipeline)
- Что пробовалось (история правок)
- Почему не удалось (предположение агента)
- Рекомендация для human

После triage_report.md: остановись, передай управление human'у.
НЕ пытайся 4-й раз. Не трать токены. Не ломай код дальше.

ВАЛИДАЦИЯ: AGENTS.md содержит разделы "## Feedback Loop" и "CIRCUIT BREAKER".
```

Пример triage_report.md

```
---
type: triage
date: 2026-06-02T14:30:00Z
attempts: 3
feature: F07-payment-gateway
---
# Triage Report

## Failure
Pipeline failed at: lint (Biome) – rule no-explicit-any

## Attempts
1. Replaced `any` with `unknown` – broke downstream type inference
2. Added `PaymentData` interface – but import cycle with types.ts
3. Refactored to generic – ESLint rule conflict (no-generics-in-props)

## Analysis
The issue is a conflict between Biome's no-explicit-any rule and
the existing type architecture. Simple replacement breaks the code,
proper refactoring requires human architectural decision.

## Recommendation
Human needed: decide whether to:
a) Disable no-explicit-any for this specific file (with ADR)
b) Refactor the type hierarchy (estimated 2-3 hours)
c) Accept `unknown` and fix all downstream consumers
```

7. Deterministic Validator (v1.1 — переписан)

ПРАВКА по ревью #1: От bash/grep к AST-парсингу

Было (v1.0): harness-validate.sh — bash с grep, wc, sed. Падает на "## Стек" вместо "## Stack", лишние пробелы, локализации.

Стало (v1.1): validate-harness.ts — Node.js + remark-parse. Проверяет AST-узлы (heading type, depth, value), не строки. + dependency-cruiser check. + JSON Schema валидация через ajv.

7.1. Архитектура валидатора v1.1

```
+-----+
| validate-harness.ts (Node.js) |
| |
| Phase 1: AST Structure (remark-parse) |
| - AGENTS.md: AST содержит heading узлы |
|   depth=2 с value из вайтлиста |
| - Instructions: frontmatter AST check |
| - Safety: list items под heading Safety |
| |
| Phase 2: Code Architecture (dep-cruiser) |
| - .dependency-cruiser.js существует |
| - npx depcruise src --config ... проходит |
| - forbidden rules определены |
| |
| Phase 3: Data Contracts (ajv + JSON Schema) |
| - feature_list.json по схеме |
| - Knowledge Base frontmatter по схеме |
| - manifest.json валиден |
| |
| Phase 4: Integration |
| - Lefthook config valid |
| - CI workflow syntax (actionlint) |
| - Linter configs parseable |
| |
| Output: manifest.json + report |
+-----+
```

7.2. Полный скрипт `validate-harness.ts`

7. Deterministic Validator (v1.1 — переписан)

```
#!/usr/bin/env tsx
/**
 * validate-harness.ts — deterministic harness validator v1.1
 * No LLM. AST-based Markdown parsing. No grep.
 */

import { unified } from 'unified';
import remarkParse from 'remark-parse';
import remarkFrontmatter from 'remark-frontmatter';
import { visit } from 'unist-util-visit';
import { readFileSync, existsSync } from 'fs';
import Ajv from 'ajv';

interface ValidationResult {
  phase: string;
  check: string;
  status: 'PASS' | 'FAIL' | 'WARN';
  detail?: string;
}

const results: ValidationResult[] = [];
let errors = 0;
let warnings = 0;

function pass(phase: string, check: string, detail?: string) {
  results.push({ phase, check, status: 'PASS', detail });
}

function fail(phase: string, check: string, detail: string) {
  results.push({ phase, check, status: 'FAIL', detail });
  errors++;
}

function warn(phase: string, check: string, detail: string) {
  results.push({ phase, check, status: 'WARN', detail });
  warnings++;
}

// =====
// PHASE 1: AST Structure (remark-parse)
// =====
function phase1_ast() {
  console.log('=== Phase 1: AST Structure ===\n');

  // 1.1 AGENTS.md exists and parses
  if (!existsSync('AGENTS.md')) {
    fail('AST', 'AGENTS.md exists', 'File not found');
    return;
  }

  const agentsContent = readFileSync('AGENTS.md', 'utf-8');
  const lines = agentsContent.split('\n').length;
  if (lines < 60) {
    pass('AST', 'AGENTS.md line count', `${lines} lines`);
  } else {
    fail('AST', 'AGENTS.md line count', `${lines} lines (max 60)`);
  }

  // Parse AST
```

```

.use(remarkParse)
  .use(remarkFrontmatter)
  .parse(agentsContent);

// Extract headings (depth=2)
const h2s: string[] = [];
visit(tree, 'heading', (node: any) => {
  if (node.depth === 2 && node.children?.[0]?.type === 'text') {
    h2s.push(node.children[0].value);
  }
});

// Check required sections (AST values, NOT grep!)
const requiredSections = [
  'Stack', 'Architecture', 'Commands',
  'Architecture Rules', 'Safety', 'Session Protocol'
];
for (const section of requiredSections) {
  const found = h2s.some(h => h.trim() === section);
  if (found) {
    pass('AST', `AGENTS.md section: ${section}`);
  } else {
    fail('AST', `AGENTS.md section: ${section}`,
      `Found headings: ${h2s.join(', ')}`);
  }
}

// 1.2 Safety section has 5+ list items
let safetyListItems = 0;
let inSafety = false;
visit(tree, (node: any) => {
  if (node.type === 'heading' && node.depth === 2 &&
    node.children?.[0]?.value === 'Safety') {
    inSafety = true;
  } else if (node.type === 'heading' && node.depth === 2) {
    inSafety = false;
  } else if (inSafety && node.type === 'listItem') {
    safetyListItems++;
  }
});
if (safetyListItems >= 5) {
  pass('AST', 'Safety rules count', `${safetyListItems} rules`);
} else {
  fail('AST', 'Safety rules count',
    `${safetyListItems} rules (need 5+)`);
}

// 1.3 Instruction Modules
if (existsSync('.claude/instructions/local') &&
  existsSync('.claude/instructions/shared')) {
  // Check local files
  const { globSync } = require('glob');
  const localFiles = globSync('.claude/instructions/local/**/*.md');
  const sharedFiles = globSync('.claude/instructions/shared/**/*.md');
  if (localFiles.length >= 2 && sharedFiles.length >= 1) {
    pass('AST', 'Instruction Modules',
      `${localFiles.length} local, ${sharedFiles.length} shared`);
  }
}

```

```

    `${localFiles.length} local, ${sharedFiles.length} shared`);
  }
} else {
  fail('AST', 'Instruction Modules dirs', 'Missing local/ or shared/');
}

// 1.4 Knowledge Base
if (existsSync('docs/knowledge/.template-decision.md') &&
  existsSync('docs/knowledge/.template-lesson.md')) {
  pass('AST', 'Knowledge Base templates');
} else {
  fail('AST', 'Knowledge Base templates', 'Missing');
}
}

// =====
// PHASE 2: Architecture (dependency-cruiser)
// =====
function phase2_architecture() {
  console.log('\n=== Phase 2: Architecture ===\n');

  if (!existsSync('.dependency-cruiser.js')) {
    fail('ARCH', '.dependency-cruiser.js exists', 'File not found');
    return;
  }

  // Check config has forbidden rules
  const config = readFileSync('.dependency-cruiser.js', 'utf-8');
  if (config.includes('forbidden') && config.includes('severity')) {
    pass('ARCH', 'dependency-cruiser config', 'Has forbidden rules');
  } else {
    fail('ARCH', 'dependency-cruiser config', 'No forbidden rules');
  }

  // Try to run depcruise (if installed)
  try {
    const { execSync } = require('child_process');
    execSync('npx depcruise src --config .dependency-cruiser.js', {
      stdio: 'pipe', timeout: 30000
    });
    pass('ARCH', 'dependency-cruiser execution', 'No violations');
  } catch (e: any) {
    if (e.stderr?.toString().includes('not found') ||
      e.message.includes('ENOENT')) {
      warn('ARCH', 'dependency-cruiser execution', 'Not installed');
    } else {
      fail('ARCH', 'dependency-cruiser execution',
        e.stderr?.toString() || e.message);
    }
  }
}

// =====
// PHASE 3: Data Contracts (JSON Schema via ajv)
// =====
function phase3_contracts() {
  console.log('\n=== Phase 3: Data Contracts ===\n');

```

```

// 3.1 feature_list.json
if (existsSync('feature_list.json') &&
    existsSync('feature_list.schema.json')) {
    try {
        const schema = JSON.parse(
            readFileSync('feature_list.schema.json', 'utf-8'));
        const data = JSON.parse(readFileSync('feature_list.json', 'utf-8'));
        const validate = ajv.compile(schema);
        if (validate(data)) {
            pass('SCHEMA', 'feature_list.json', 'Valid per schema');
        } else {
            fail('SCHEMA', 'feature_list.json',
                ajv.errorsText(validate.errors));
        }
    } catch (e: any) {
        fail('SCHEMA', 'feature_list.json', e.message);
    }
} else {
    fail('SCHEMA', 'feature_list.json', 'Missing file or schema');
}

// 3.2 Knowledge Base frontmatter
if (existsSync('docs/knowledge/.schema.json')) {
    pass('SCHEMA', 'Knowledge Base schema', 'Exists');
} else {
    warn('SCHEMA', 'Knowledge Base schema', 'Missing .schema.json');
}
}

// =====
// PHASE 4: Integration
// =====
function phase4_integration() {
    console.log('\n=== Phase 4: Integration ===\n');

    // 4.1 Lefthook
    if (existsSync('lefthook.yml')) {
        const content = readFileSync('lefthook.yml', 'utf-8');
        const cmdCount = (content.match(/\s+run:/gm) || []).length;
        if (cmdCount >= 3) {
            pass('INT', 'lefthook.yml', `${cmdCount} commands`);
        } else {
            fail('INT', 'lefthook.yml', `${cmdCount} commands (need 3+)`);
        }
    } else {
        fail('INT', 'lefthook.yml', 'Missing');
    }

    // 4.2 CI workflow
    if (existsSync('.github/workflows/ci.yml')) {
        const content = readFileSync('.github/workflows/ci.yml', 'utf-8');
        const jobCount = (content.match(/runs-on:/g) || []).length;
        if (jobCount >= 4) { // lint, typecheck, test, architecture
            pass('INT', 'CI workflow', `${jobCount} jobs (incl. architecture)`);
        } else {
            fail('INT', 'CI workflow', `${jobCount} jobs (need 4+)`);
        }
    }
}

```

7. Deterministic Validator (v1.1 — переписан)

```
    fail('INT', 'CI workflow', 'Missing');
  }

  // 4.3 Linter config
  if (existsSync('biome.json') || existsSync('eslint.config.js') ||
    existsSync('pyproject.toml')) {
    pass('INT', 'Linter config', 'Present');
  } else {
    fail('INT', 'Linter config', 'Missing');
  }
}

// =====
// GENERATE MANIFEST
// =====
function generateManifest() {
  const manifest = {
    validated_at: new Date().toISOString(),
    version: '1.1',
    errors,
    warnings,
    results
  };

  const fs = require('fs');
  fs.mkdirSync('.harness', { recursive: true });
  fs.writeFileSync('.harness/manifest.json',
    JSON.stringify(manifest, null, 2));
  console.log('\n=== Manifest ===');
  console.log('Written to .harness/manifest.json');
}

// =====
// MAIN
// =====
console.log('=====');
console.log(' AI Harness Validator v1.1');
console.log(' AST-based | dep-cruiser | JSON Schema');
console.log('=====');

phase1_ast();
phase2_architecture();
phase3_contracts();
phase4_integration();
generateManifest();

console.log('\n=====');
console.log(` ERRORS:   ${errors}`);
console.log(` WARNINGS: ${warnings}`);
console.log('=====');

process.exit(errors > 0 ? 1 : 0);
```

7.3. package.json для валидатора

```
{
  "name": "@your-org/harness-validator",
  "version": "1.1.0",
  "bin": { "harness-validate": "./validate-harness.ts" },
  "scripts": { "validate": "tsx validate-harness.ts" },
  "dependencies": {
    "unified": "^11.0.0",
    "remark-parse": "^11.0.0",
    "remark-frontmatter": "^5.0.0",
    "unist-util-visit": "^5.0.0",
    "ajv": "^8.12.0",
    "glob": "^10.3.0"
  },
  "devDependencies": {
    "tsx": "^4.7.0",
    "typescript": "^5.4.0",
    "@types/node": "^20.0.0"
  }
}
```

8. JSON Schema контракты

PРАВКА по ревью #4: Строгая валидация метаданных

Было (v1.0): `head -1 | grep "---"` — проверял только разделитель frontmatter.

Стало (v1.1): JSON Schema для `feature_list.json` (через `ajv-cli`) и frontmatter Knowledge Base. Проверка типов, enum значений, форматов дат.

8.1. feature_list.schema.json

```

{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "$id": "https://your-org.github.io/harness/feature_list.schema.json",
  "title": "Feature List",
  "type": "object",
  "required": ["project", "features", "tech_debt", "last_updated"],
  "additionalProperties": false,
  "properties": {
    "project": {
      "type": "string",
      "minLength": 1,
      "description": "Project name from package.json"
    },
    "features": {
      "type": "array",
      "items": {
        "type": "object",
        "required": ["id", "title", "status"],
        "additionalProperties": false,
        "properties": {
          "id": {
            "type": "string",
            "pattern": "^F[0-9]{2}$",
            "description": "Feature ID: F01, F02, ..."
          },
          "title": {
            "type": "string",
            "minLength": 3,
            "maxLength": 100
          },
          "status": {
            "type": "string",
            "enum": ["pending", "in_progress", "completed", "blocked"]
          },
          "commit": {
            "type": "string",
            "pattern": "^[a-f0-9]{7,40}$"
          }
        }
      }
    },
    "tech_debt": {
      "type": "array",
      "items": { "type": "string", "minLength": 5 }
    },
    "active_decisions": {
      "type": "array",
      "items": {
        "type": "object",
        "required": ["adr_ref", "title"],
        "properties": {
          "adr_ref": { "type": "string", "pattern": "^ADR-[0-9]{3}$" },
          "title": { "type": "string" }
        }
      }
    },
    "last_updated": {

```

```

"format": "date"
  }
}
}

```

8.2. knowledge_frontmatter.schema.json

```

{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "$id": "https://your-org.github.io/harness/knowledge.schema.json",
  "title": "Knowledge Base Entry Frontmatter",
  "type": "object",
  "required": ["type", "tags", "date"],
  "additionalProperties": false,
  "properties": {
    "type": {
      "type": "string",
      "enum": ["decision", "lesson", "pattern"],
      "description": "Entry type"
    },
    "tags": {
      "type": "array",
      "minItems": 1,
      "items": {
        "type": "string",
        "pattern": "^[a-z][a-z0-9-]+$"
      }
    },
    "date": {
      "type": "string",
      "format": "date"
    },
    "status": {
      "type": "string",
      "enum": ["accepted", "deprecated", "superseded"],
      "default": "accepted"
    },
    "severity": {
      "type": "string",
      "enum": ["high", "medium", "low"]
    }
  },
  "allOf": [
    {
      "if": { "properties": { "type": { "const": "decision" } } },
      "then": { "required": ["status"] }
    },
    {
      "if": { "properties": { "type": { "const": "lesson" } } },
      "then": { "required": ["severity"] }
    }
  ]
}

```

8.3. Использование валидации

```
# Валидация feature_list.json
npx ajv-cli validate \
  --schema=feature_list.schema.json \
  --errors=text \
  feature_list.json

# Валидация frontmatter Knowledge Base (через remark-frontmatter + ajv)
npx tsx validate-knowledge.ts docs/knowledge/

# В lefthook (прекоммит хук):
# ajv-cli validate --schema=feature_list.schema.json feature_list.json
```

9. Манифест (.harness/manifest.json)

9.1. Структура

```
{
  "validated_at": "2026-06-02T14:30:00Z",
  "version": "1.1",
  "skill_version": "1.2.0",
  "errors": 0,
  "warnings": 0,
  "generated_by": "claude-sonnet-4",
  "results": [
    { "phase": "AST", "check": "AGENTS.md: Stack section",
      "status": "PASS" },
    { "phase": "ARCH", "check": "dependency-cruiser config",
      "status": "PASS", "detail": "Has forbidden rules" },
    { "phase": "SCHEMA", "check": "feature_list.json",
      "status": "PASS", "detail": "Valid per schema" }
  ],
  "project": {
    "stack": "nextjs-react-ts",
    "detected_from": "package.json"
  },
  "modules": {
    "agents_md": { "file": "AGENTS.md", "lines": 42,
      "sections": ["Stack", "Architecture", "Commands",
        "Architecture Rules", "Safety", "Session Protocol"] },
    "quality_pipeline": { "lefthook": true, "ci": true,
      "dependency_cruiser": true,
      "jobs": ["lint", "typecheck", "test", "architecture", "build"] },
    "session_protocol": { "feature_list": true,
      "schema_valid": true },
    "knowledge_base": { "templates": 3,
      "schema": "knowledge_frontmatter.schema.json" }
  }
}
```

10. Три режима работы

10.1. Interactive Setup

1. Агент читает SKILL.md
2. Агент сканирует проект (package.json, структура)
3. Агент задает уточняющие вопросы
4. Пользователь отвечает
5. Агент генерирует артефакты (AGENTS.md, lefthook.yml, ...)
6. Агент копирует templates (.template-*.md, .dependency-cruiser.js)
7. Агент запускает: npx tsx validate-harness.ts
8. Validator создает .harness/manifest.json
9. Агент выводит: "Harness configured. Errors: 0. Warnings: X."

10.2. Sync

1. Новая версия SKILL.md
2. Агент читает diff
3. Агент показывает изменения, пользователь подтверждает
4. Агент применяет изменения
5. Validator проверяет
6. Manifest обновляется

10.3. CI-only (deterministic, без LLM)

```
.github/workflows/harness-validate.yml:
name: Harness Validation
on: [push, pull_request]
jobs:
  validate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
      - run: npm ci # установка зависимостей валидатора
      - run: npx tsx validate-harness.ts
      # FAIL if exit code != 0
```


11. Пример: полный SKILL.md v1.1

11. Пример: полный SKILL.md v1.1

```
# AI Harness Skill v1.1
# Гибридный: skill описывает, агент генерирует, AST+Schema валидирует.

## Принципы
1. Я описываю модули декларативно (что нужно).
2. Ты генерируешь артефакты (как – зависит от стека).
3. Security-критичное копируется из templates/.
4. После генерации запускаешь: npx tsx validate-harness.ts.
5. Если validator падает – чини и перезапускай (max 3 попытки).

## Setup Questions
1. "Стек проекта? [auto-detect from package.json]"
2. "SSR? [auto-detect]"
3. "Monorepo или single repo? [auto-detect]"
4. "Shared instruction registry? [URL или skip]"
5. "Требования безопасности? [default / HIPAA / PCI]"

---

## Модуль: AGENTS.md
НЕОБХОДИМО: AGENTS.md в корне, < 50 строк.
ШАБЛОН: template/AGENTS.md

СТРУКТУРА:
## Stack
[из package.json]
## Architecture
[layered | monolith | mfe]
## Commands
[из scripts: dev, build, test, lint, typecheck]
## Architecture Rules
Enforced by .dependency-cruiser.js – see config.
## Safety
[фиксированный список из 5+ запретов]
## Session Protocol
[протокол сессии]
## Feedback Loop
[цикл обратной связи + Circuit Breaker]

ВАЛИДАЦИЯ (AST, НЕ grep):
- remark-parse: heading "Stack", "Commands", "Architecture Rules",
  "Safety", "Session Protocol"
- .dependency-cruiser.js существует и содержит forbidden[]

---

## Модуль: Quality Pipeline
НЕОБХОДИМО: lefthook.yml + .dependency-cruiser.js + CI workflow.

ИНСТРУМЕНТЫ (из стека):
TypeScript -> Biome + ESLint + tsc + vitest + dependency-cruiser

LOCAL (lefthook.yml):
parallel: lint, typecheck, test, architecture
architecture: depcruise src --config .dependency-cruiser.js
```

```

TEMPLATE: lefthook.yml, ci.yml – из template/, замени {VARIABLES}

---

## Модуль: Knowledge Base
НЕОБХОДИМО: docs/knowledge/ с шаблонами и .schema.json.

ШАБЛОНЫ: копируй из template/knowledge/
JSON SCHEMA: docs/knowledge/.schema.json (frontmatter validation)

---

## Модуль: Session Protocol
ТИП: Embedded + feature_list.json

JSON SCHEMA: feature_list.schema.json (ajv validation)
ВАЛИДАЦИЯ: npx ajv-cli validate --schema=feature_list.schema.json feature_list.json

---

## Модуль: Feedback Loop + Circuit Breaker
ТИП: Process (раздел в AGENTS.md)

CIRCUIT BREAKER:
  ATTEMPTS_LIMIT = 3
  После 3 неудач: triage_report.md + STOP + human handoff

---

## Post-Setup
1. npx tsx validate-harness.ts
2. Если ошибки – исправь (итерация 1). Повтори validator.
3. Если опять ошибки – исправь (итерация 2). Повтори validator.
4. Если опять – triage_report.md и остановись.
5. После успеха: git add -A && git commit -m "harness: setup v1.0"

```

Итого изменений v1.0 -> v1.1

4 модуля обновлены по результатам аудита. Validator полностью переписан (bash -> TypeScript + AST). Добавлены: dependency-cruiser для Architecture-as-Code, Circuit Breaker для Feedback Loop, JSON Schema для Data Contracts. Риски генерации сведены к минимуму: 5 модулей (из 8) не генерируются агентом.